



**TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ ĐÔNG Á**
EAST ASIA UNIVERSITY OF TECHNOLOGY

KHOA CÔNG NGHỆ THÔNG TIN

KIỂM THỬ PHẦN MỀM

Chương 4: Phân tích và thiết kế test



1. Hiểu được vai trò của phân tích và thiết kế test
2. Nhận biết các bước chính trong quá trình phân tích và thiết kế test
3. Làm quen với các kỹ thuật thiết kế test phổ biến
4. Biết cách lựa chọn kỹ thuật phù hợp
5. Phát triển kỹ năng viết test case rõ ràng, hiệu quả và có thể tái sử dụng.

4.1 Tổng quan về kỹ thuật thiết kế test

4.2 Kỹ thuật kiểm thử hộp đen

4.3 Kỹ thuật kiểm thử hộp trắng

4.4 Kỹ thuật kiểm thử dựa trên kinh nghiệm

4.1 Tổng quan về kỹ thuật thiết kế test

- Kỹ thuật thiết kế test (test design techniques) là bộ công cụ tư duy giúp tester xác định các test case một cách có hệ thống và hiệu quả, giúp tester:
 - Phân tích (what to test): cần hiểu rõ hệ thống có những chức năng nào, dữ liệu nào cần kiểm thử, và đâu là các yếu tố quan trọng trong logic nghiệp vụ
 - Thiết kế test (how to test): thiết kế test case phù hợp để thực hiện điều đó. Bạn cần lựa chọn kỹ thuật test: sử dụng phân vùng tương đương, giá trị biên, hay kỹ thuật dựa vào bảng quyết định.

4.1 Tổng quan về kỹ thuật thiết kế test

- Phát triển một tập các test-case đầy đủ, có hệ thống. Một tập test-case đầy đủ không chỉ giúp bao phủ hết các điều kiện đầu vào mà còn đảm bảo bạn có đủ dữ liệu kiểm thử để kiểm tra tất cả các trường hợp logic, trạng thái và hành vi của hệ thống
- Xác định các điều kiện kiểm thử, độ bao phủ và dữ liệu kiểm thử. Mỗi test case phải tương ứng với một mục tiêu cụ thể: một yêu cầu, một điều kiện logic, một dòng dữ liệu hoặc một kịch bản người dùng.

4.1 Tổng quan về kỹ thuật thiết kế test

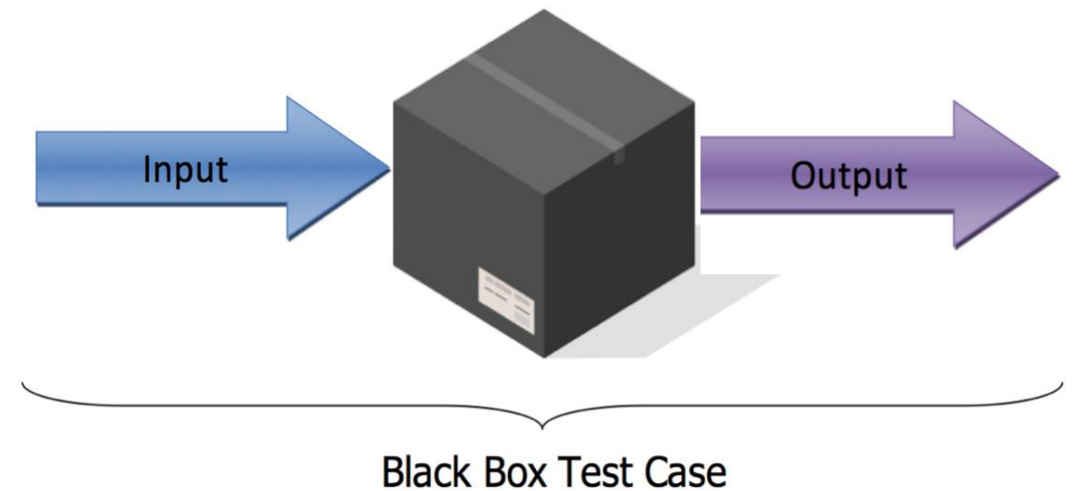
- Phân loại
 - Kỹ thuật test hộp đen (Black-box test techniques)
 - Kỹ thuật test hộp trắng (White-box test techniques)
 - Kỹ thuật test dựa trên kinh nghiệm (Experience-based test techniques)

4.1 Tổng quan về kỹ thuật thiết kế test

Kỹ thuật kiểm thử hộp đen (hay còn được gọi là kỹ thuật kiểm thử dựa vào đặc tả - specification-based testing) Là phương pháp test dựa trên đầu vào và đầu ra của chương trình để test mà không quan tâm tới code bên trong được viết ra sao. Phương pháp này thường dùng để test chức năng của chương trình.

Phương pháp:

- Dựa trên phân tích hành vi cụ thể của đối tượng test mà không tham chiếu đến cấu trúc bên trong của nó
- Các testcase độc lập với cách triển khai phần mềm



4.1 Tổng quan về kỹ thuật thiết kế test

- Kiểm thử dựa trên kinh nghiệm (Experience-based test techniques)
 - Phương pháp: Sử dụng kiến thức, kỹ năng và kinh nghiệm của tester để thiết kế và triển khai testcase. Ngoài việc tuân thủ tài liệu (như requirement, spec), còn có những tình huống tài liệu không rõ ràng, không chi tiết hoặc thậm chí không có tài liệu.

4.1 Tổng quan về kỹ thuật thiết kế test

Kiểm thử dựa trên kinh nghiệm (Experience-based test techniques) : Khi không có tài liệu hoặc tài liệu không rõ ràng, tester kỳ cựu sẽ sử dụng:

- Kiến thức về dự án tương tự (đã test trước đó).
- Kiến thức về công nghệ (như web, mobile, backend).
- Kiến thức về người sử dụng (user behavior).
- Kiến thức về các vấn đề hay phát sinh trong phần mềm đó.

4.1 Tổng quan về kỹ thuật thiết kế test

Kiểm thử dựa trên kinh nghiệm (Experience-based test techniques) Khi làm test dựa trên kinh nghiệm, tester sẽ:

- Ước lượng nhanh phần nào dễ có vấn đề.
- Kiểm thử bằng cách đặt mình vào vai người sử dụng (user).
- Áp dụng các kỹ thuật test tự do (exploratory testing).
- Kiểm thử theo kiểu tấn công phần mềm bằng cách nhập thử dữ liệu ngoài ý muốn, thử tải cao, thử trên nhiều trình duyệt, nhiều mạng khác nhau.
- Kiểm thử bằng danh sách vấn đề quen thuộc (checklist) tích lũy được từ nhiều dự án trước.

4.1 Tổng quan về kỹ thuật thiết kế test

4.2 Kỹ thuật kiểm thử hộp đen

4.3 Kỹ thuật kiểm thử hộp trắng

4.4 Kỹ thuật kiểm thử dựa trên kinh nghiệm

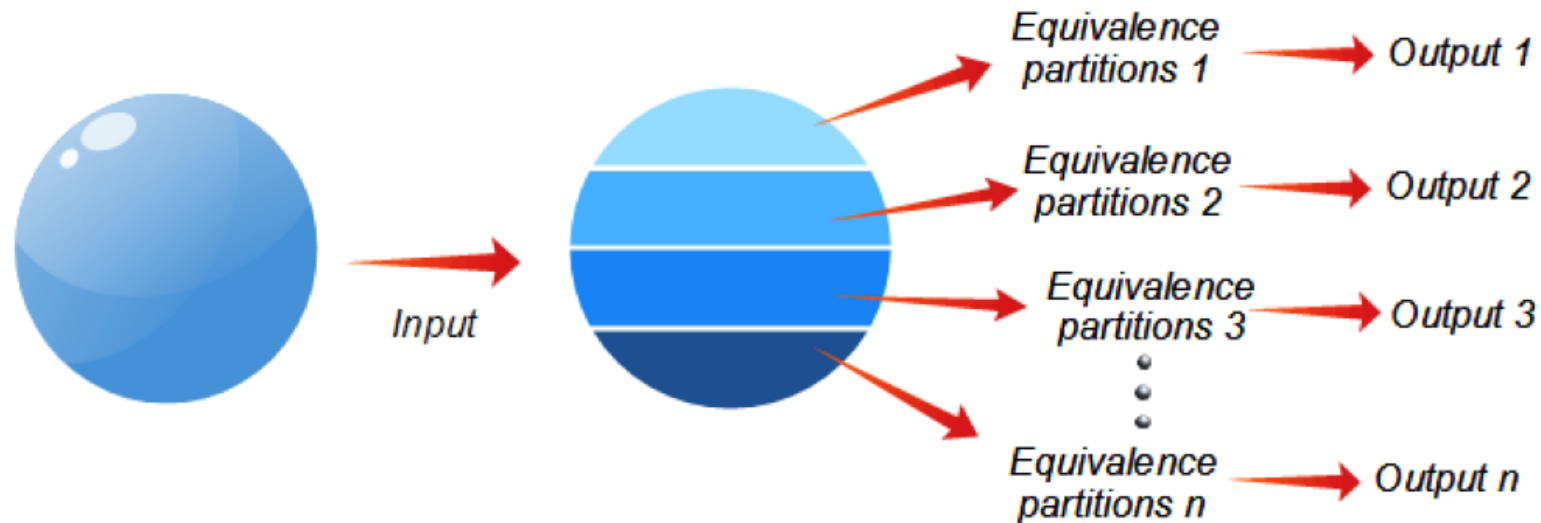
4.2 Kỹ thuật kiểm thử hộp đen

Các kỹ thuật kiểm thử hộp đen phổ biến:

- Phân vùng tương đương (Equivalence Partitioning)
- Phân tích giá trị biên (Boundary Value Analysis)
- Bảng quyết định (Decision Table Testing)
- Chuyển trạng thái (State Transition Testing)

4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

Phương pháp: Là phương pháp chia dữ liệu thành các vùng sao cho tất cả các giá trị trong một vùng sẽ có kết quả đầu ra giống nhau (vùng tương đương)



4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)



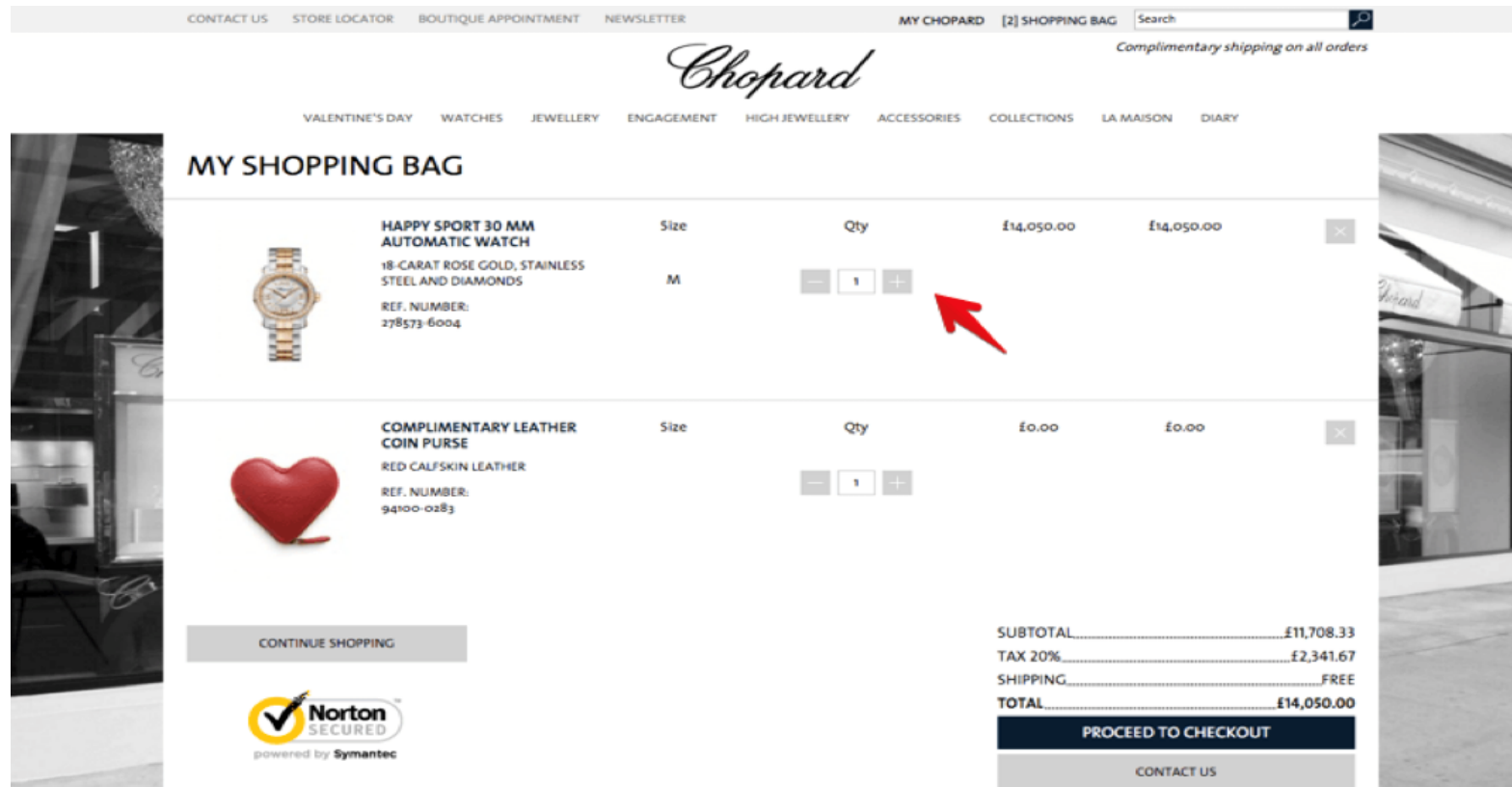
Xác định test:

Nếu một trường hợp kiểm thử kiểm tra một giá trị từ một phân vùng tương đương, phát hiện ra lỗi thì lỗi này cũng sẽ được phát hiện bởi các trường hợp kiểm thử kiểm tra bất kỳ giá trị nào khác từ cùng một phân vùng.

=> Vì vậy, một test cho mỗi phân vùng là đủ.

4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

Ví dụ: Dựa trên kỹ thuật EP, thiết kế testcase cho màn hình giỏ hàng với yêu cầu sau



- Mỗi sản phẩm khách hàng chỉ được mua với số lượng không quá 100.
- Nếu số lượng trên 10 thì được chiết khấu 5%.
- Ngược lại thì không được chiết khấu.

4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

Xác định phân vùng

Invalid		Valid		Valid		Invalid
0		1	10	11	99	100
Partition 1		Partition 2		Partition 3		Partition 4

4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

2. Xác định testcase

TC	Giá trị Input	Kết quả Output mong đợi
1	0 (phân vùng $(-\infty, 0]$)	Hiển thị thông báo Invalid number
2	5 (phân vùng $[1, 10]$)	Total = Quantity * Unit price
3	50 (phân vùng $[11, 99]$)	Total = Quantity * Unit price - Discount
4	100 (phân vùng $[100, +\infty)$)	Hiển thị thông báo Invalid number

4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

Ưu điểm

Mỗi vùng tương đương chỉ cần test trên các phần tử đại diện nên số lượng TC giảm -> giảm thời gian viết TC -> giảm thời gian test.

4.2.1 Phân vùng tương đương (Equivalence Partitioning - EP)



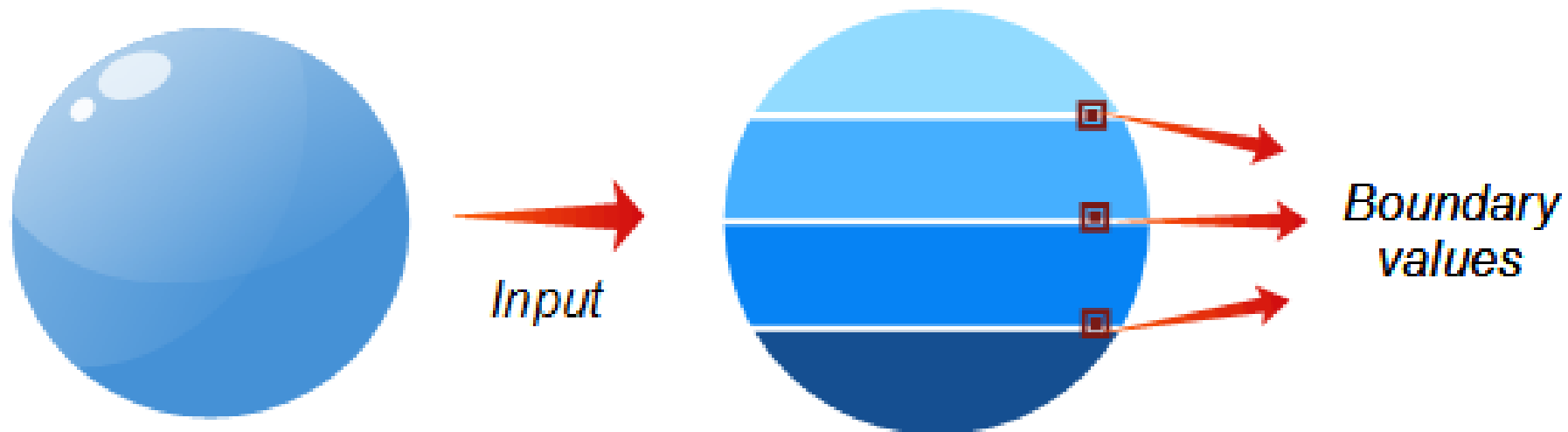
Nhược điểm

Không phát hiện được lỗi ở các giá trị ranh giới giữa các phân vùng.

4.2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

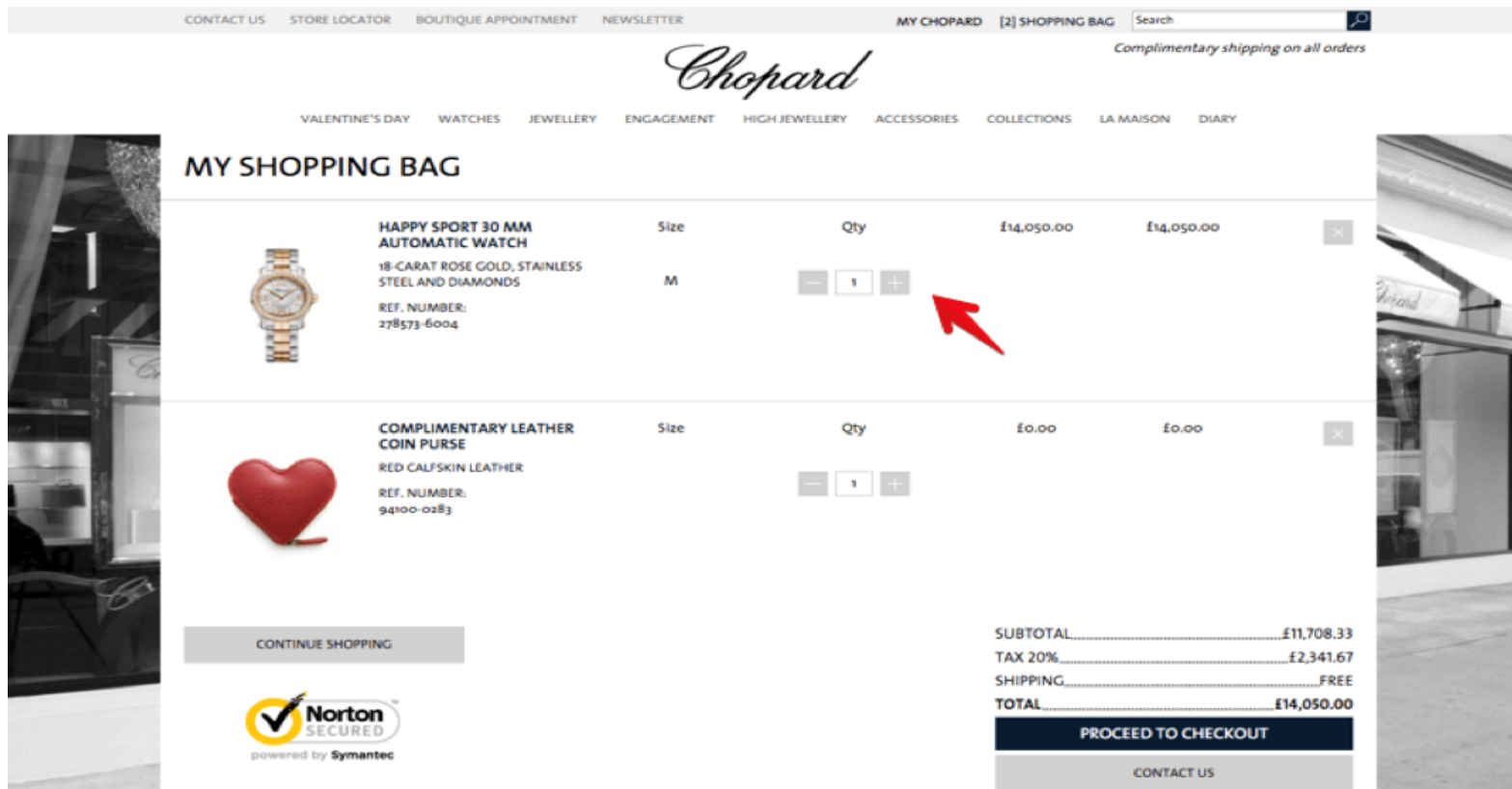
Phương pháp Là tiến trình lựa chọn test data bằng cách tìm hiểu ranh giới giữa các phân vùng.

Min và Max của các phân vùng là giá trị biên của nó.



4.2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Ví dụ: Dựa trên kỹ thuật EP, thiết kế testcase cho màn hình giỏ hàng với yêu cầu sau



- Mỗi sản phẩm khách hàng chỉ được mua với số lượng không quá 100.
- Nếu số lượng trên 10 thì được chiết khấu 5%.
- Ngược lại thì không được chiết khấu.

4.2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

1. Xác định các điểm biên

Phân vùng	Giá trị biên
$(-\infty, 0]$	0
$[1, 10]$	1, 10
$[11, 99]$	11, 99
$[100, +\infty)$	100

4.2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

2. Xác định testcase

TC	Giá trị Input	Kết quả mong đợi
1	0	Hiển thị thông báo Invalid number
2	1	Total = Quantity * Unit price
3	10	Total = Quantity * Unit price
4	11	Total = Quantity * Unit price - Discount
5	99	Total = Quantity * Unit price - Discount
6	100	Hiển thị thông báo Invalid number

4.2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

- Ưu điểm: Phát hiện được lỗi tiềm ẩn của hệ thống tại các tập hợp biên
- Nhược điểm: Nếu chỉ chọn giá trị biên thì không đảm bảo chương trình chạy đúng với các giá trị ở khoảng giữa.

Kết hợp EP và BVA



Bộ test data = $A \cup B$

A: Bộ test data của các phân vùng tương đương

B: Bộ test data của biên

Gửi tiền tiết kiệm ở ngân hàng với các mức lãi suất khác nhau tùy thuộc vào số tiền gửi:

- Gửi từ \$0 đến \$100 thì hưởng lãi suất 3%
- Gửi từ trên \$100 đến \$1000, lãi suất là 5%
- Gửi từ trên \$1000 trở lên, lãi suất là 7%

Hãy xác định TC sử dụng 2 phương pháp phân vùng tương đương và phân tích giá trị biên.

4.2.3 Bảng quyết định (Decision Table Testing)

- Bảng quyết định được sử dụng để kiểm tra việc triển khai các yêu cầu hệ thống nhằm xác định cách kết hợp các điều kiện khác nhau sẽ dẫn đến các kết quả khác nhau.
- Bảng quyết định là một cách hiệu quả để ghi lại logic phức tạp, chẳng hạn như các business rule.

4.2.3 Bảng quyết định (Decision Table Testing)

Thành phần của bảng quyết định

	R1	R2	R3	R4	R5	R6	R7	R8	...
C1	T	T	-	T	F	F	F	F	
C2	T	T	F	F	T	T	F	F	
C3	T	F	T	F	T	F	T	F	
...									
a1	x			x	x				
a2		x				x	x		
a3			x	x					

Rules

Conditions

Values of conditions

Actions taken

4.2.3 Bảng quyết định (Decision Table Testing)

Giải thích ký hiệu:

“T” (True) có nghĩa là điều kiện đó được thỏa mãn

“F” (False) có nghĩa là điều kiện không được thỏa mãn

“–” có nghĩa là giá trị của điều kiện không liên quan đến kết quả hành động

Đối với hành động: “X” có nghĩa là hành động đó sẽ xảy ra

Trống có nghĩa là hành động đó không nên xảy ra

4.2.3 Bảng quyết định (Decision Table Testing)

Phương pháp:

B1. Liệt kê toàn bộ điều kiện đầu vào/Input

B2. Tính toán số lượng các kết hợp (Rules)

B3. Đặt toàn bộ các kết hợp vào bảng

B4. Giảm số lượng các kết hợp và quyết định test case.

4.2.3 Bảng quyết định (Decision Table Testing)

Ví dụ: Xác định testcase cho bài toán khách hàng đến mở thẻ tín dụng

Nếu bạn là một khách hàng mới, đến mở thẻ tín dụng, bạn sẽ được giảm giá 15%.

Nếu bạn là khách hàng cũ, và có thẻ Vip, bạn sẽ được giảm giá 10%.

Nếu bạn có Coupon, bạn sẽ được giảm giá 20% (nhưng nó không được sử dụng giảm giá cùng với 'khách hàng mới'.

Việc giảm giá có thể được cộng nếu như phù hợp

4.2.3 Bảng quyết định (Decision Table Testing)



B1: Liệt kê giá trị đầu vào

Conditions
Khách hàng mới (15%)
VIP card (10%)
Coupon (20%)

4.2.3 Bảng quyết định (Decision Table Testing)

B2: Xác định số rules

	R1	R2	R3	R4	R5	R6	R7	R8
Khách hàng mới (15%)								
VIP card (10%)								
Coupon (20%)								

4.2.3 Bảng quyết định (Decision Table Testing)

B3:Đặt toàn bộ các kết hợp vào bảng

	R1	R2	R3	R4	R5	R6	R7	R8
Khách hàng mới (15%)	T	T	T	T	F	F	F	F
VIP card (10%)	T	T	F	F	T	T	F	F
Coupon (20%)	T	F	T	F	T	F	T	F
Giảm giá 10%	-	-			X	X		
Giảm giá 15%	-	-		X				
Giảm giá 20%	-	-	X		X		X	
Không giảm giá	-	-						X

4.2.3 Bảng quyết định (Decision Table Testing)

B4:Rút gọn bảng

	R1	R2	R3	R4	R5	R6	R7	R8
Khách hàng mới (15%)	T	T	T	T	F	F	F	F
VIP card (10%)	T	T	F	F	T	T	F	F
Coupon (20%)	T	F	T	F	T	F	T	F
Giảm giá 10%	-	-			x	x		
Giảm giá 15%	-	-		x				
Giảm giá 20%	-	-	x?		x		x	
Không giảm giá	-	-						x

4.2.3 Bảng quyết định (Decision Table Testing)

B4:Rút gọn bảng

Nhìn vào Bảng quyết định ta có thể thấy:

- R1 và R2 là 2 case ko có trong thực tế nên loại bỏ.
- R3 và R7 có cùng kết quả nên ta có thể bỏ R7

Như vậy, ta có thể chọn các TC như sau: R3, R4, R5, R6, R7.

4.2.3 Bảng quyết định (Decision Table Testing)

- Ưu điểm:
 - Mô tả các quy tắc nghiệp vụ phức tạp dưới dạng dễ đọc và dễ kiểm soát
 - Xác định số test case tối thiểu với độ bao phủ tối đa
- Nhược điểm:
 - Nếu có quá nhiều kết hợp, chúng ta sẽ ko thể test tất cả các kết hợp đó

4.2.3 Bảng quyết định (Decision Table Testing)



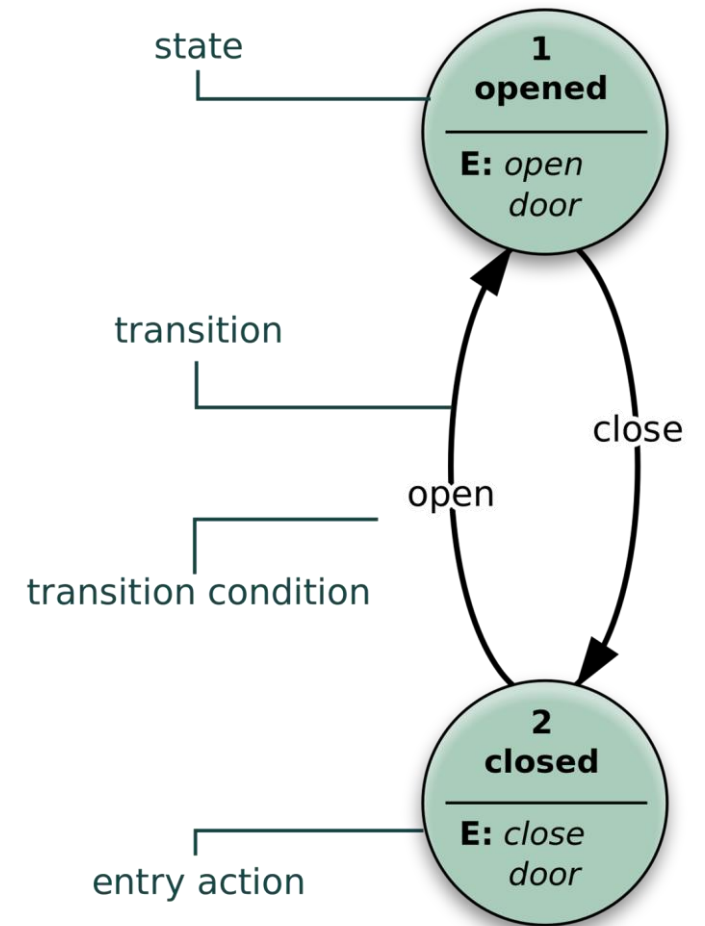
Bài tập

- Nếu bạn có thẻ railcard "over 60s - hơn 60 tuổi" thì được giảm giá 34% trên tất cả các vé bạn mua.
- Nếu bạn đi cùng với trẻ em (dưới 16 tuổi) và có thẻ Family railcard, thì bạn sẽ được giảm 50%.
- Nếu bạn có thẻ Family railcard và không đi cùng trẻ em, bạn được giảm 10%.
- Bạn chỉ được sử dụng 1 loại thẻ railcard



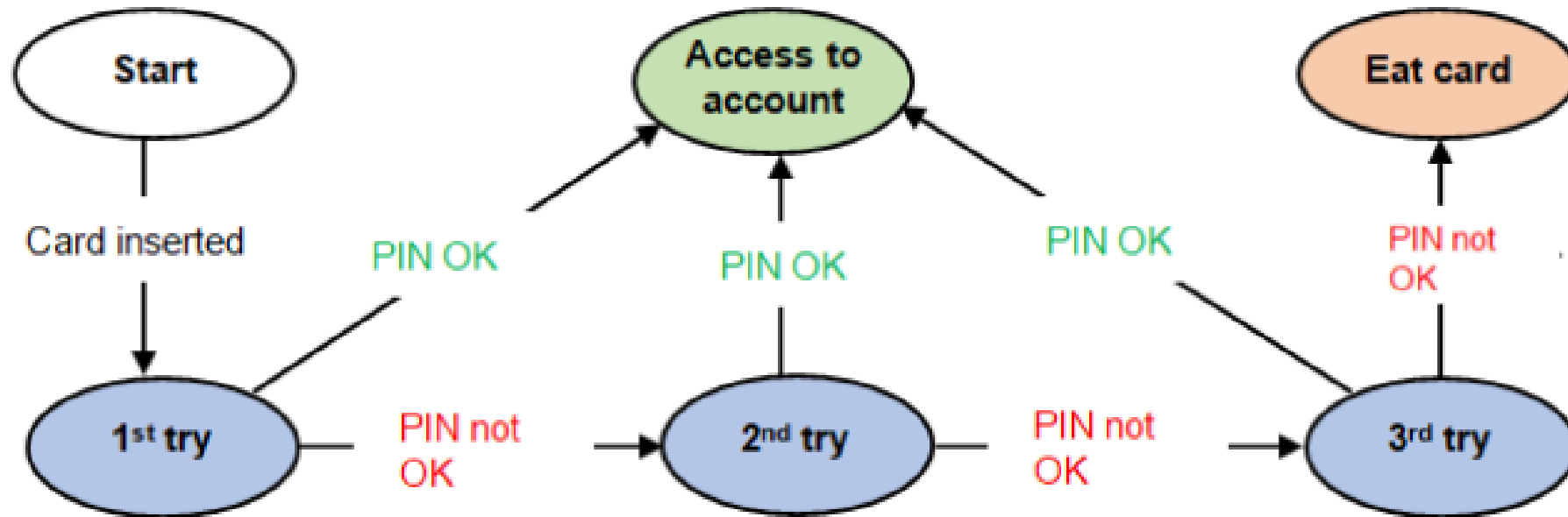
4.2.4 Chuyển trạng thái (State Transition Testing)

- Sơ đồ chuyển trạng thái mô hình hóa hành vi của hệ thống bằng cách hiển thị các trạng thái có thể có và trạng thái chuyển tiếp hợp lệ.
- Quá trình chuyển đổi được bắt đầu bởi một sự kiện hoặc một điều kiện.



4.2.4 Chuyển trạng thái (State Transition Testing)

Ví dụ: Sơ đồ chuyển trạng thái của chức năng login trong hệ thống ATM



4.2.4 Chuyển trạng thái (State Transition Testing)

Tuy nhiên chuyển trạng thái cũng có thể được mô tả bằng bảng

	Sự kiện/Điều kiện	
	Events/Condition 1	Events/Condition 2
Trạng thái	S1	S2
	S2	-
	S3	S3
		...

Trạng thái có thể đạt được (nếu không thể đạt được thì để -)

4.2.4 Chuyển trạng thái (State Transition Testing)

Ví dụ: Bảng chuyển trạng thái của chức năng login trong hệ thống ATM

	Insert Card	Valid PIN	Invalid PIN
S1 (Start)	S2	-	-
S2 (1st try)	-	S6	S3
S3 (2nd try)	-	S6	S4
S4 (3rd try)	-	S6	S5
S5 (Eat Card)	S1		
S6 (Access to Account)			

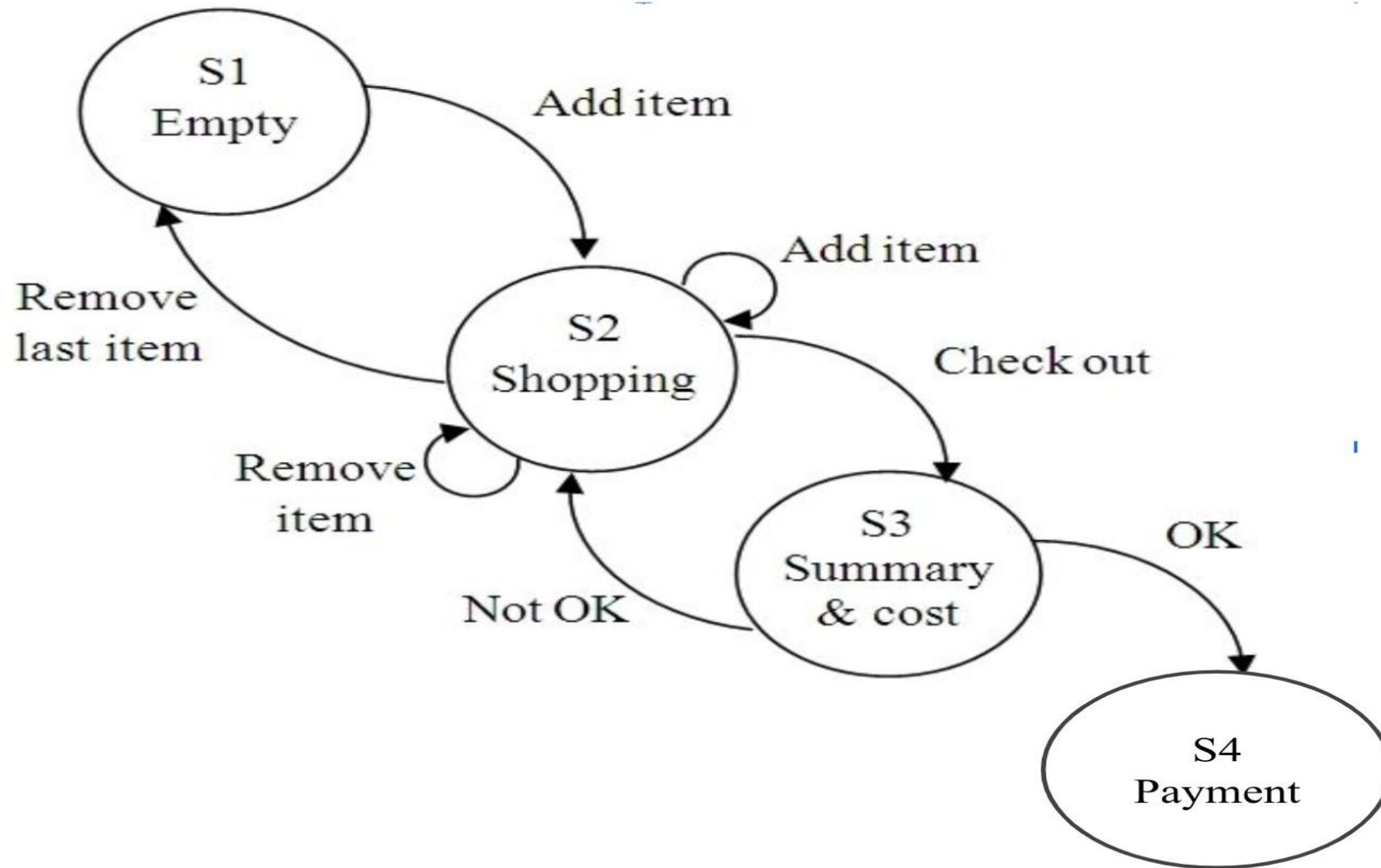
4.2.4 Chuyển trạng thái (State Transition Testing)

- Một test case dựa trên sơ đồ chuyển trạng thái hoặc bảng trạng thái thường được biểu diễn dưới dạng một chuỗi các sự kiện dẫn đến một chuỗi các thay đổi trạng thái (và các hành động, nếu cần).
 - Một test case thường sẽ bao gồm một số chuyển đổi giữa các trạng thái.
 - Độ bao phủ:
 - Độ bao phủ các states (Tổng số test case để bao phủ 100% các states)
 - Độ bao phủ các transitions (Tổng số test case để bao phủ 100% các transactions)
 - Độ bao phủ n-switch (đi qua n-1 trạng thái)
- => Với chức năng Login trong hệ thống ATM cần
4 test case để cover 100% transactions
2 test case để cover 100% states

4.2.4 Chuyển trạng thái (State Transition Testing)

Bài tập:

- Giỏ hàng trên một trang mua bán trực tuyến được bắt đầu với trạng thái là rỗng (không có món hàng nào).
- Khi bạn chọn một sản phẩm thì nó sẽ được đưa vào giỏ hàng. Bạn cũng có thể bỏ chọn các món hàng trong giỏ hàng.
- Khi bạn quyết định mua hàng, thì sẽ xuất hiện màn hình tổng hợp các món hàng đang có trong giỏ cùng với thông tin về giá tiền, số lượng và tổng tiền của giỏ hàng, để cho bạn xác nhận xem đúng hay chưa.
- Nếu bạn thấy số lượng hàng và giá tiền OK thì bạn sẽ được chuyển sang trang thanh toán. Ngược lại bạn sẽ quay lại trang mua hàng (lúc này bạn có thể bỏ chọn các món hàng bạn muốn bỏ bớt).
- Yêu cầu:
- Đưa ra sơ đồ trạng thái - state diagram – cho thấy các trạng thái/states và sự chuyển tiếp/transition khác.
- Xác định test case bao phủ toàn bộ các trạng thái.
- Xác định test case bao phủ toàn bộ các chuyển tiếp.



4.1 Tổng quan về kỹ thuật thiết kế test

4.2 Kỹ thuật kiểm thử hộp đen

4.3 Kỹ thuật kiểm thử hộp trắng

4.4 Kỹ thuật kiểm thử dựa trên kinh nghiệm

4.3 Kỹ thuật kiểm thử hộp trắng (White-Box Test Techniques)

Các kỹ thuật liên quan đến kiểm thử hộp trắng:

- Kiểm thử dòng lệnh (Statement testing)
- Kiểm thử nhánh (Branch testing)

4.3.1 Kiểm thử dòng lệnh và độ bao phủ dòng lệnh

- Mục đích là thiết kế các ca kiểm thử thực hiện các câu lệnh trong mã cho đến khi đạt được mức độ bao phủ có thể chấp nhận được.
- Mức độ bao phủ được đo bằng: $(\text{số lượng câu lệnh được thực hiện bởi testcase} / \text{tổng số các câu lệnh có thể thực thi được trong mã}) * 100\%$

4.3.1 Kiểm thử dòng lệnh và độ bao phủ dòng lệnh

Ví dụ: Xác định testcase tối thiểu để đạt được độ bao phủ 100% các dòng lệnh

1	READ p;
2	READ q;
3	IF p+q > 100 THEN
4	PRINT "Large" ;
5	ENDIF

Số Testcase	Input	Expected Output
1	p = 60 q = 50	Print "Large"
Số dòng lệnh là 5 Với 1 bộ test case trên có thể đạt được 100% độ bao phủ dòng lệnh		

4.3.1 Kiểm thử dòng lệnh và độ bao phủ dòng lệnh

Bài tập: Xác định testcase tối thiểu để đạt được độ bao phủ 100% các dòng lệnh

```
READ A
READ B
IF A < 0 THEN
    PRINT "A negative"
ELSE
    PRINT "A positive"
ENDIF
IF B < 0 THEN
    PRINT "B negative"
ELSE
    PRINT "B positive"
ENDIF
```

4.3.1 Kiểm thử dòng lệnh và độ bao phủ dòng lệnh

=> Khi đạt được mức độ bao phủ câu lệnh 100%, nó đảm bảo rằng tất cả các câu lệnh thực thi trong mã đã được thực hiện ít nhất một lần.

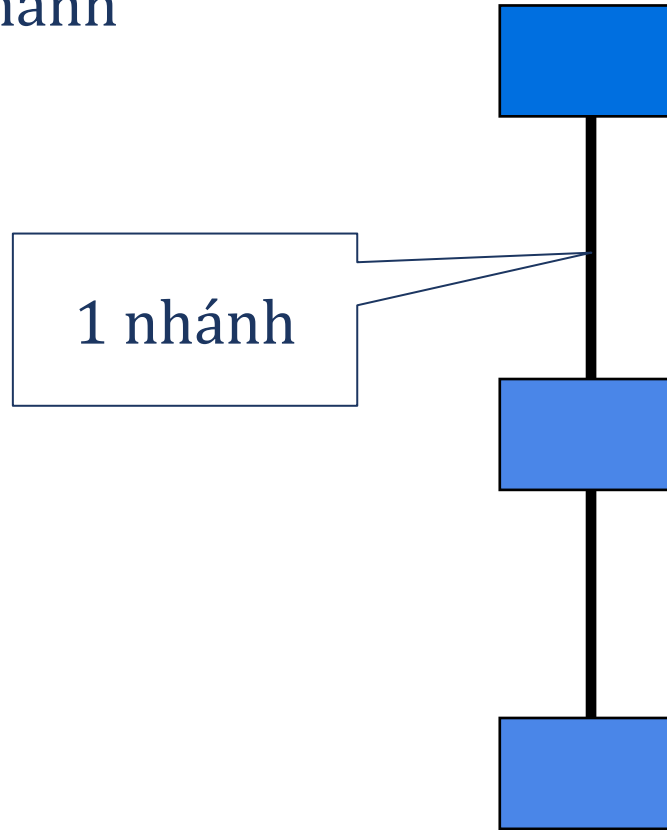
=> Tuy nhiên kiểm thử dòng lệnh sẽ không phát hiện được lỗi trong mọi trường hợp đặc biệt liên quan đến quyết định logic

4.3.2 Kiểm thử nhánh và độ bao phủ nhánh

- Nhánh là sự chuyển giao quyền điều khiển giữa hai nút trong biểu đồ luồng điều khiển, cho thấy khả năng trình tự các câu lệnh mã nguồn được thực thi trong đối tượng kiểm thử.
- Mỗi lần chuyển giao quyền kiểm soát có thể là vô điều kiện (tức là mã đường thẳng) hoặc có điều kiện (tức là kết quả quyết định).
- Các nhánh có điều kiện thường tương ứng với kết quả đúng hoặc sai từ một quyết định “if...then”, kết quả từ câu lệnh “switch/case” hoặc quyết định thoát hoặc tiếp tục vòng lặp “loop”

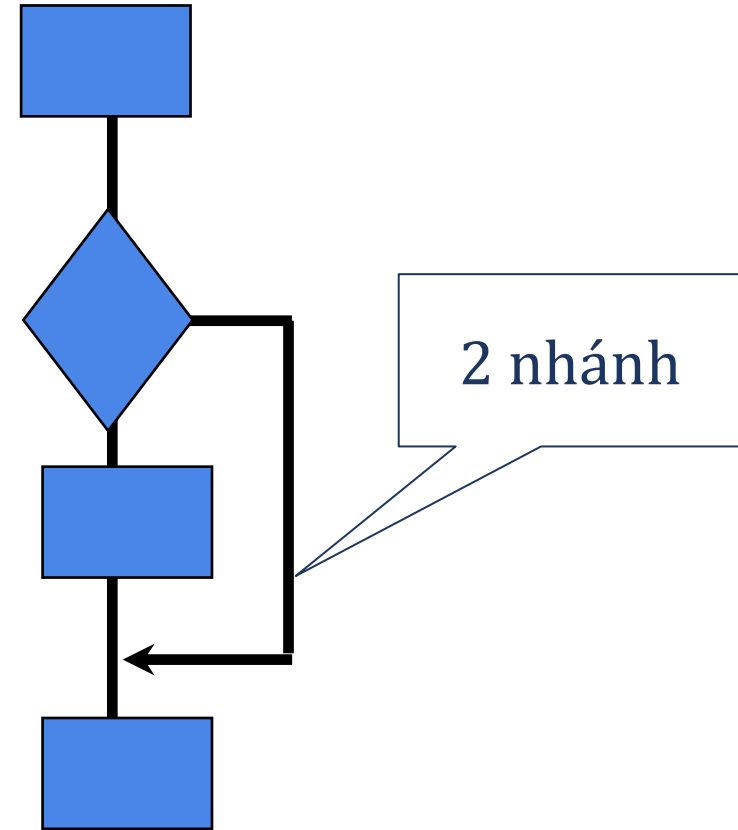
4.3.2 Kiểm thử nhánh và độ bao phủ nhánh

Nhánh



1 nhánh

Chuyển giao vô
điều kiện



2 nhánh

Chuyển giao có
điều kiện

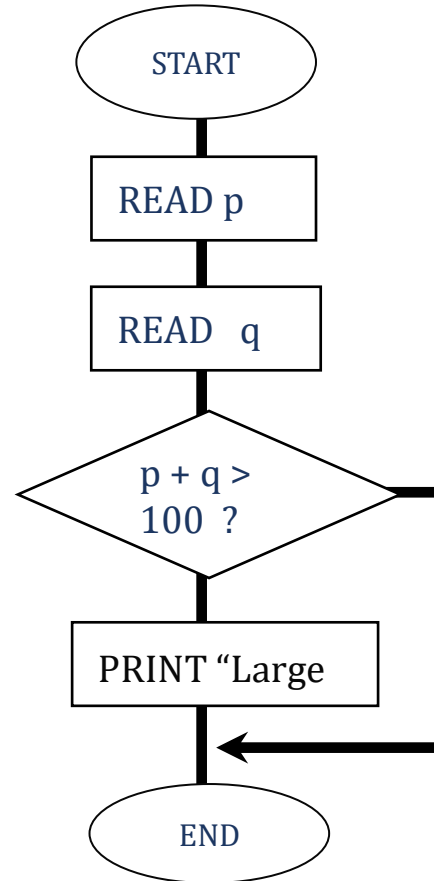
4.3.2 Kiểm thử nhánh và độ bao phủ nhánh

- Mục đích là thiết kế các Test Case để thực hiện các nhánh trong mã cho đến khi đạt được mức độ bao phủ có thể chấp nhận được.
- Mức độ bao phủ được đo bằng: $(\text{số nhánh được thực hiện bởi các testcase} / \text{tổng số nhánh}) / 100\%$

4.3.2 Kiểm thử nhánh và độ bao phủ nhánh

Ví dụ: Xác định testcase tối thiểu để đạt được độ bao phủ 100% nhánh

1	READ p;
2	READ q;
3	IF p+q > 100 THEN
4	PRINT "Large" ;
5	ENDIF



Số Testcase	Input	Expected Output
2	p = 60 q = 50	Print "Large"
	p = 60 q = 5	Do nothing

Số nhánh là 2

Với 2 bộ test case trên có thể đạt được 100% độ bao phủ nhánh

4.3.2 Kiểm thử nhánh và độ bao phủ nhánh

Khi đạt được phạm vi bao phủ 100% nhánh, tất cả các nhánh trong mã, vô điều kiện và có điều kiện, đều được thực hiện bằng các Test Case.

=> Bao phủ nhánh cover bao phủ dòng lệnh. Điều này có nghĩa là bất kỳ bộ test nào đạt được 100% bao phủ nhánh cũng đạt được 100% bao phủ dòng lệnh (nhưng không ngược lại).

4.1 Tổng quan về kỹ thuật thiết kế test

4.2 Kỹ thuật kiểm thử hộp đen

4.3 Kỹ thuật kiểm thử hộp trắng

4.4 Kỹ thuật kiểm thử dựa trên kinh nghiệm

4.4 Kỹ thuật kiểm thử dựa trên kinh nghiệm

Các kỹ thuật liên quan đến kiểm thử dựa trên kinh nghiệm:

- Dự đoán lỗi (Error guessing)
- Kiểm thử khám phá (Exploratory testing)
- Kiểm thử dựa trên checklist (Checklist-based testing)

4.4.1 Dự đoán lỗi

- "Error guessing" là một kỹ thuật được sử dụng để dự đoán sự xuất hiện của errors, defects và failure dựa trên kiến thức của người kiểm thử, bao gồm:
 - Cách ứng dụng đã hoạt động trong quá khứ
 - Các loại lỗi mà dev thường mắc phải và các loại lỗi phát sinh từ những lỗi này
 - Các loại failures đã xảy ra trong các ứng dụng tương tự khác

4.4.1 Dự đoán lỗi



- Kỹ thuật này đòi hỏi người kiểm thử tạo ra hoặc thu thập danh sách các errors, defects và failures có thể xảy ra, đồng thời thiết kế các test case có khả năng phát hiện ra các defects.
- Những danh sách lỗi này có thể được xây dựng dựa trên kinh nghiệm, dữ liệu lỗi và lỗi hoặc từ kiến thức chung về lý do tại sao phần mềm bị lỗi.

4.4.1 Dự đoán lỗi

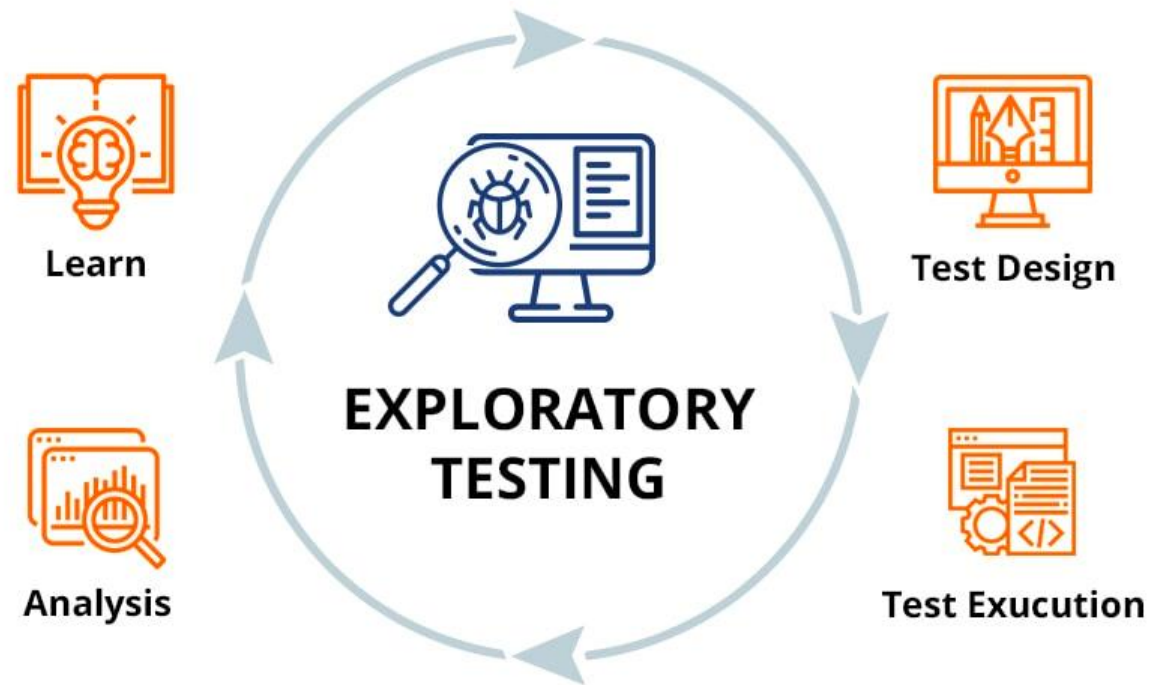


Một số lỗi

- input (đúng input không được chấp nhận, sai hoặc thiếu các parameters)
- output (sai format, sai result)
- logic (thiếu case, vận hành sai)
- computation (sai toán hạng, tính toán sai)
- interfaces (parameter không khớp hoặc không tương thích)
- data (sai khởi tạo, sai kiểu)

4.4.2 Kiểm thử khám phá

- Các test case được thiết kế, thực thi và đánh giá đồng thời trong quá trình tester tìm hiểu về đối tượng kiểm thử



4.4.2 Kiểm thử khám phá

- Được thực hiện trong một khoảng thời gian xác định (session-based testing).
- Tester sử dụng một bảng kiểm thử (test charter) chứa các mục tiêu kiểm thử để hướng dẫn quá trình kiểm thử (họ có thể sử dụng các bảng ghi chú phiên làm việc để ghi lại các bước thực hiện và các khám phá đã được thực hiện).
- Sau phiên kiểm thử tester và các bên liên quan sẽ trao đổi về kết quả kiểm thử

4.4.2 Kiểm thử khám phá



Ví dụ test charter:

Charter-011

Explore class list page
with various inputs
for the assessment setting
to discover interactions between
class list and
the assessment setting

4.4.2 Kiểm thử khám phá

- Kiểm thử khám phá rất hữu ích khi có ít hoặc không đầy đủ tài liệu đặc tả hoặc áp lực về thời gian thực hiện
- Kiểm thử khám phá sẽ hiệu quả hơn nếu người kiểm thử có kinh nghiệm, có kiến thức nghiệp vụ và có trình độ cao về các kỹ năng thiết yếu, như kỹ năng phân tích, tính tò mò và tính sáng tạo

4.4.3 Kiểm thử dựa trên checklist

- Trong checklist-based testing, tester thiết kế, xây dựng, và thực hiện test để bao phủ các test conditions từ checklist
- Các mục của checklist thường là những question đề cập đến các yêu cầu, giao diện, chất lượng phần mềm
- Checklist được xây dựng dựa trên kinh nghiệm, kiến thức về những gì là quan trọng đối với người dùng, hoặc hiểu biết tại sao và như thế nào phần mềm bị lỗi

4.4.3 Kiểm thử dựa trên checklist

Ví dụ checklist:

Question	Yes	No	N/A	Note	Priority	Severity
Is graphical design of GUI reviewed or accepted by customer? If not					Mandatory	1
Has the waiver been stated?					Mandatory	1
Is one button set to be default?						4
Are the controls ordered according to normal behavior (Microsoft Style)?					Optional	4
Are all labels aligned uniformly?					Optional	4
Are text areas aligned uniformly?					Optional	4
Are there any spelling mistakes?					Mandatory	4
Are all non-enterable text areas of the same color (gray)?					Optional	4
Is the tab sequence correct?					Mandatory	4
Is cursor becomes hourglass when doing background processing?					Optional	3
Does cursor return to pointer after the task?					Optional	3
Do all fields match data length?					Optional	4
Do enterable fields accept entry only for the given lengths of String e.g. char (30)?					Optional	4
Are abbreviations used? If yes					Optional	4
Are they understandable for end users?					Optional	4
Are they consistently used across all screens?					Optional	4
Are both mouse and key strokes handled?					Optional	3
Are the color combinations used on screens consistent?					Optional	4
Is the usage of the screens intuitive?					Optional	4
If hotkeys or accelerator keys used, is it consistent?					Optional	4
If yes to the above, has this been tested?					Mandatory	3
Are error messages displayed in non-technical terms and are understandable to the user?					Optional	3
Are the error messages informative and not blaming the user?					Mandatory	4
Before an action that affects the system (such as a delete) is performed, is the user asked for confirmation?					Mandatory	2
Are dates and time formats across the system consistent?					Optional	4
Are number formats consistent?					Optional	4
If Tooltips are used, are they helpful or meaningful?					Optional	4
Are screens and menu mapped correctly?					Mandatory	2

4.5 Quiz

